

Prompt: Task 1: Fix the coin-collection bug in my Pygame Coin Collector game. In game/game_engine.py, update() calls check_collection every frame and adds the coin's value to the score, but the coin is never removed, so walking through one coin scores it a dozen times. Fix it so each coin is collected exactly once. Give me the complete updated file, not a diff.

Response: Here's the complete updated game/game_engine.py. The fix is in update(): collected coins are now removed from the list, so they can only score once.

```
"""
GameEngine: owns the player and all coins.
Starter version: one coin type, no obstacles, no timer yet.
Task 1 fix: collected coins are removed from the list, so each coin
is collected exactly once.
"""
import random
import pygame
from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
NUM_COINS = 6
COIN_VALUE = 1

class GameEngine:

    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.score = 0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        return Coin(x=x, y=y, radius=12, value=COIN_VALUE)

    def handle_input(self, keys_pressed):
        dx = dy = 0
        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed
        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed
        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed
        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed
        self.player.move(dx, dy, WIDTH, HEIGHT)

    def update(self):
        collected = check_collection(self.player, self.coins)
        for coin in collected:
            self.score += coin.value
            self.coins.remove(coin) # Task 1 fix: collect each coin only once

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.player, self.coins)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

Prompt: Task 2: Add multiple coin types to my Pygame Coin Collector game. I need at least three types: bronze (1 point), silver (3 points), and gold (5 points). Each type needs its own color so they're visually distinguishable, and the correct value must be awarded when each is collected. Gold should be rarer than bronze. Tell me which files change and give me the complete updated file(s), not diffs.

Response: Only one file changes: game/game_engine.py. Coin already accepts a value and a color, so coin.py and renderer.py stay as they are.

```
"""
GameEngine: owns the player and all coins.
Task 1 fix: collected coins are removed from the list, so each coin
```

```

is collected exactly once.
Task 2: three coin types (bronze, silver, gold) with different values,
colors, and spawn rarity.
"""
import random
import pygame
from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
NUM_COINS = 6
# (name, value, color, spawn weight) - higher weight = more common
COIN_TYPES = [
    ("bronze", 1, (205, 127, 50), 60),
    ("silver", 3, (192, 192, 192), 30),
    ("gold", 5, (255, 215, 0), 10),
]

class GameEngine:

    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.score = 0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        weights = [t[3] for t in COIN_TYPES]
        _, value, color, _ = random.choices(COIN_TYPES, weights=weights, k=1)[0]
        return Coin(x=x, y=y, radius=12, value=value, color=color)

    def handle_input(self, keys_pressed):
        dx = dy = 0
        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed
        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed
        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed
        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed
        self.player.move(dx, dy, WIDTH, HEIGHT)

    def update(self):
        collected = check_collection(self.player, self.coins)
        for coin in collected:
            self.score += coin.value
            self.coins.remove(coin) # Task 1 fix: collect each coin only once

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.player, self.coins)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

```

Prompt: Task 3: Add obstacles to my Pygame Coin Collector game. The player must avoid them while collecting coins. Colliding with an obstacle should cost the player 1 life (start with 3 lives) and show the lives on screen. A single touch must not drain lives every frame while still overlapping, so add a short invincibility period after a hit. Obstacles must stay inside the play area, must not spawn on top of the player or on coins, and should be visually distinct (e.g. red squares). Tell me which files change and give me the complete updated file(s), not diffs.

Response: Two files change and one new file is added:

New: game/obstacle.py

Changed: game/game_engine.py

Changed: game/renderer.py

File 1 (new): game/obstacle.py

```

"""
Obstacle: a static hazard square. Touching it costs the player a life.
"""
import pygame

class Obstacle:

    def __init__(self, x, y, size=34):
        self.x = x
        self.y = y
        self.size = size

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.size / 2), int(self.y - self.size / 2),
            self.size, self.size,
        )

```

File 2: game/game_engine.py (full replacement)

```

"""
GameEngine: owns the player, coins, and obstacles.
Task 1: collected coins are removed, so each is collected exactly once.
Task 2: three coin types (bronze, silver, gold).
Task 3: obstacles cost a life, with a short invincibility window after
a hit so one touch doesn't drain lives every frame.
"""
import random
import pygame
from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
NUM_COINS = 6
NUM_OBSTACLES = 4
START_LIVES = 3
INVINCIBLE_FRAMES = 90 # 1.5 seconds at 60 FPS
# (name, value, color, spawn weight) - higher weight = more common
COIN_TYPES = [
    ("bronze", 1, (205, 127, 50), 60),
    ("silver", 3, (192, 192, 192), 30),
    ("gold", 5, (255, 215, 0), 10),
]

class GameEngine:

    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.obstacles = self._spawn_obstacles()
        self.score = 0
        self.lives = START_LIVES
        self.invincible_frames = 0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        weights = [t[3] for t in COIN_TYPES]
        _, value, color, _ = random.choices(COIN_TYPES, weights=weights, k=1)[0]
        return Coin(x=x, y=y, radius=12, value=value, color=color)

    def _spawn_obstacles(self):
        """Place obstacles inside the play area, away from the player,
        the coins, and each other."""
        obstacles = []
        safe_zone = self.player.get_rect().inflate(160, 160)
        attempts = 0
        while len(obstacles) < NUM_OBSTACLES and attempts < 500:
            attempts += 1
            size = 34
            margin = size // 2 + 5
            x = random.randint(margin, WIDTH - margin)

```

```

        y = random.randint(margin, HEIGHT - margin)
        candidate = Obstacle(x, y, size)
        rect = candidate.get_rect()
        if rect.colliderect(safe_zone):
            continue
        if any(rect.colliderect(c.get_rect().inflate(10, 10)) for c in self.coins):
            continue
        if any(rect.colliderect(o.get_rect().inflate(20, 20)) for o in obstacles):
            continue
        obstacles.append(candidate)
    return obstacles

def handle_input(self, keys_pressed):
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed
    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed
    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed
    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed
    self.player.move(dx, dy, WIDTH, HEIGHT)

def update(self):
    # Coins (Task 1 + 2)
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
    # Invincibility countdown
    if self.invincible_frames > 0:
        self.invincible_frames -= 1
    # Obstacles (Task 3): lose one life per hit, then become invincible
    if self.invincible_frames == 0 and self.lives > 0:
        player_rect = self.player.get_rect()
        for obstacle in self.obstacles:
            if player_rect.colliderect(obstacle.get_rect()):
                self.lives -= 1
                self.invincible_frames = INVINCIBLE_FRAMES
                break

def draw(self, surface, font):
    from game import renderer
    # Blink the player while invincible
    blink_hidden = (
        self.invincible_frames > 0
        and (pygame.time.get_ticks() // 100) % 2 == 0
    )
    renderer.draw_scene(
        surface, self.player, self.coins, self.obstacles, hide_player=blink_hidden
    )
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))

```

File 3: game/renderer.py (full replacement)

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame
WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)
COLOR_BG = (35, 45, 35)
COLOR_PLAYER = (80, 180, 255)
COLOR_OBSACLE = (220, 60, 60)
COLOR_OBSACLE_BORDER = (120, 20, 20)
COLOR_TEXT = (255, 255, 255)

def draw_scene(surface, player, coins, obstacles=(), hide_player=False):
    surface.fill(COLOR_BG)
    for coin in coins:
        pygame.draw.circle(surface, coin.color, (int(coin.x), int(coin.y)), coin.radius)
    for obstacle in obstacles:

```

```

        rect = obstacle.get_rect()
        pygame.draw.rect(surface, COLOR_OBSACLE, rect)
        pygame.draw.rect(surface, COLOR_OBSACLE_BORDER, rect, width=3)
    if not hide_player:
        pygame.draw.rect(surface, COLOR_PLAYER, player.get_rect(), border_radius=4)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)

```

Prompt: Task 4: Add a timed round to my Pygame Coin Collector game. Add a 30-second countdown and display the remaining time on screen. The round ends when time reaches zero OR lives reach zero, whichever comes first. When it ends, stop gameplay (player can't move, no more scoring or life loss), show a clear "Game Over" screen with the final score, and let me press R to start a new round that resets score, lives, timer, coins, and obstacles. Tell me which files change and give me the complete updated file(s), not diffs.

Response: For Task 4, three files change: main.py (to catch the R key), game/game_engine.py, and game/renderer.py.

File 1: game/renderer.py (full replacement)

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""
import pygame
WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)
COLOR_BG = (35, 45, 35)
COLOR_PLAYER = (80, 180, 255)
COLOR_OBSACLE = (220, 60, 60)
COLOR_OBSACLE_BORDER = (120, 20, 20)
COLOR_TEXT = (255, 255, 255)
_big_font = None

def draw_scene(surface, player, coins, obstacles=(), hide_player=False):
    surface.fill(COLOR_BG)
    for coin in coins:
        pygame.draw.circle(surface, coin.color, (int(coin.x), int(coin.y)), coin.radius)
    for obstacle in obstacles:
        rect = obstacle.get_rect()
        pygame.draw.rect(surface, COLOR_OBSACLE, rect)
        pygame.draw.rect(surface, COLOR_OBSACLE_BORDER, rect, width=3)
    if not hide_player:
        pygame.draw.rect(surface, COLOR_PLAYER, player.get_rect(), border_radius=4)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)

def draw_game_over(surface, font, score, reason):
    """Dark overlay with Game Over, the reason, final score, and restart hint."""
    global _big_font
    if _big_font is None:
        _big_font = pygame.font.SysFont("consolas", 52, bold=True)
    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 170))
    surface.blit(overlay, (0, 0))
    cx = surface.get_width() // 2
    cy = surface.get_height() // 2
    lines = [
        (_big_font, "GAME OVER", (255, 90, 90), cy - 80),
        (font, reason, (220, 220, 220), cy - 25),

```

```

        (_big_font, f"Final Score: {score}", (255, 220, 80), cy + 25),
        (font, "Press R to play again", (255, 255, 255), cy + 90),
    ]
    for fnt, text, color, y in lines:
        surf = fnt.render(text, True, color)
        surface.blit(surf, surf.get_rect(center=(cx, y)))

```

File 2: game/game_engine.py (full replacement)

```

"""
GameEngine: owns the player, coins, obstacles, and round state.
Task 1: collected coins are removed, so each is collected exactly once.
Task 2: three coin types (bronze, silver, gold).
Task 3: obstacles cost a life, with a short invincibility window.
Task 4: 30-second round; ends on time up or 0 lives; press R to restart.
"""

import math
import random
import pygame
from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
FPS = 60
NUM_COINS = 6
NUM_OBSTACLES = 4
START_LIVES = 3
INVINCIBLE_FRAMES = 90 # 1.5 seconds at 60 FPS
ROUND_SECONDS = 30
# (name, value, color, spawn weight) - higher weight = more common
COIN_TYPES = [
    ("bronze", 1, (205, 127, 50), 60),
    ("silver", 3, (192, 192, 192), 30),
    ("gold", 5, (255, 215, 0), 10),
]

class GameEngine:

    def __init__(self):
        self.reset()

    def reset(self):
        """Start a fresh round: score, lives, timer, coins, obstacles."""
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.obstacles = self._spawn_obstacles()
        self.score = 0
        self.lives = START_LIVES
        self.invincible_frames = 0
        self.frames_left = ROUND_SECONDS * FPS
        self.game_over = False
        self.game_over_reason = ""

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        weights = [t[3] for t in COIN_TYPES]
        _, value, color, _ = random.choices(COIN_TYPES, weights=weights, k=1)[0]
        return Coin(x=x, y=y, radius=12, value=value, color=color)

    def _spawn_obstacles(self):
        """Place obstacles inside the play area, away from the player,
        the coins, and each other."""
        obstacles = []
        safe_zone = self.player.get_rect().inflate(160, 160)
        attempts = 0
        while len(obstacles) < NUM_OBSTACLES and attempts < 500:
            attempts += 1
            size = 34
            margin = size // 2 + 5
            x = random.randint(margin, WIDTH - margin)
            y = random.randint(margin, HEIGHT - margin)
            candidate = Obstacle(x, y, size)

```

```

        rect = candidate.get_rect()
        if rect.colliderect(safe_zone):
            continue
        if any(rect.colliderect(c.get_rect().inflate(10, 10)) for c in self.coins):
            continue
        if any(rect.colliderect(o.get_rect().inflate(20, 20)) for o in obstacles):
            continue
        obstacles.append(candidate)
    return obstacles

def handle_event(self, event):
    """Press R after the round ends to start a new one."""
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r and self.game_over:
        self.reset()

def handle_input(self, keys_pressed):
    if self.game_over:
        return
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed
    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed
    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed
    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed
    self.player.move(dx, dy, WIDTH, HEIGHT)

def update(self):
    if self.game_over:
        return
    # Coins (Task 1 + 2)
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
    # Invincibility countdown
    if self.invincible_frames > 0:
        self.invincible_frames -= 1
    # Obstacles (Task 3): lose one life per hit, then become invincible
    if self.invincible_frames == 0:
        player_rect = self.player.get_rect()
        for obstacle in self.obstacles:
            if player_rect.colliderect(obstacle.get_rect()):
                self.lives -= 1
                self.invincible_frames = INVINCIBLE_FRAMES
                break
    # Timer (Task 4)
    self.frames_left -= 1
    # Round end: lives out or time up, whichever comes first
    if self.lives <= 0:
        self.game_over = True
        self.game_over_reason = "You ran out of lives!"
    elif self.frames_left <= 0:
        self.frames_left = 0
        self.game_over = True
        self.game_over_reason = "Time's up!"

def draw(self, surface, font):
    from game import renderer
    # Blink the player while invincible
    blink_hidden = (
        self.invincible_frames > 0
        and not self.game_over
        and (pygame.time.get_ticks() // 100) % 2 == 0
    )
    renderer.draw_scene(
        surface, self.player, self.coins, self.obstacles, hide_player=blink_hidden
    )
    seconds_left = math.ceil(self.frames_left / FPS)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))

```

```
        renderer.draw_text(surface, font, f"Time: {seconds_left}", (WIDTH - 130, 10))
    if self.game_over:
        renderer.draw_game_over(surface, font, self.score, self.game_over_reason)
```

File 3: main.py (full replacement)

```
"""
```

Coin Collector (Lab Starter)

Run with: python3 main.py

Controls: Arrow keys to move. Press R to restart after Game Over.

```
"""
```

```
import pygame
from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Coin Collector")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)
    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)
        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)
        pygame.display.flip()
        clock.tick(60)
    pygame.quit()
if __name__ == "__main__":
    main()
```